

Wave 1 Implementation Audit: Decoupled LogQ Deployment (1B $\rightarrow$ 1A)

DAMPS-MMHCL Phase 2 — Section 8.2 Compliance Verification
Code-Level vs. Execution-Level Completion Assessment

DAMPS-MMHCL Research Team

June 15, 2026

Contents

1	Executive Summary	2
2	Issues Preventing Wave 1 From Reaching 100% “Completion”	2
2.1	Issue 1 — The training notebook never enables LogQ (the largest gap)	2
2.2	Issue 2 — No evidence of a 10/10 green test run	2
2.3	Issue 3 — No backward-compatibility smoke test	2
2.4	Issue 4 — The <code>logq_scale</code> sweep has not been run	2
2.5	Issue 5 — A minor technical risk inside the LogQ code path	2
3	Compliance Matrix (Code Level)	3
4	Summary	3
5	Appendix: M1.5 Sweep Driver Notebook Cell	3

1 Executive Summary

The 1B/1A source code achieves **100% conformance** to the design specification — including the subtle safety details (fail-fast behaviour, item-branch-only application, and float32 overflow protection). However, while the implementation is complete at the code level, **Wave 1 is not yet 100% “complete” at the execution level**, because the M1.5 milestone has never actually been run. The following sections detail the gap.

Completion Dimension	Level	Status
Code-level (1B + 1A implementation)	~100%	✓ Spec-conformant, incl. safety details
Execution-level (M1.5 milestone)	Not met	! No run, no logs, no sweep

2 Issues Preventing Wave 1 From Reaching 100% “Completion”

2.1 Issue 1 — The training notebook never enables LogQ (the largest gap)

The notebook `Local_Random_seeds_train_mmhcl_clothing` currently runs only the rev44 Phase 1 configuration (`apc_off_combined`). A full inspection of every `train.py` invocation reveals only nine `--damps_*` flags (`apc`, `avrfl`, `imcf`, `fft`, `routing`, `momentum`, `prior`, `categories`, `warmup`) and **not a single `--enable_logq` line** (zero occurrences of the token `logq` anywhere in the notebook). This means the Wave 1 code has been merged but **never executed** — the M1.5 milestone has no data.

2.2 Issue 2 — No evidence of a 10/10 green test run

The M1.5 README mandates that “all tests must be green before merging 1A,” but the repository contains no CI artifact or pytest log. The output of `pytest tests/test_popularity_prior.py -v` must be attached as evidence.

2.3 Issue 3 — No backward-compatibility smoke test

The README requires running one epoch with `--enable_logq 0` and then performing a bit-for-bit `diff` against rev45. No log demonstrating that this check was performed has been found.

2.4 Issue 4 — The `logq_scale` sweep has not been run

The specification stresses that `logq_scale` $\in \{0.05, 0.1, 0.3, 1.0\}$ must be swept *before* trusting the literal specification, because `scale = 1.0` produces logits of magnitude ~ 40 , which overflow. This sweep is a precondition of the M1.5 gate and is currently absent.

2.5 Issue 5 — A minor technical risk inside the LogQ code path

In the `use_logq` branch, `col_slice` is passed as the *entire* indices for every chunk, whereas `refl_sim/between_sim` are computed over `z1[mask]` (the current chunk only). It must be confirmed that `log_q_term[col_slice]` is applied correctly along the column axis (the full N -item dimension) and is not misaligned when `batch_size < num_nodes`. The current logic appears correct (columns = the full item set), but a small numerical test should verify the loss value when $N > \text{batch size}$.

Table 1: Code-level conformance of the 1B + 1A implementation against rev54 §8.2.

Spec requirement (rev54 §8.2)	Actual implementation	Status
Split 1B (estimator) from 1A (loss) into two PRs	damps/popularity_prior.py is pure; loss patch is separate	✓
$q(i)$ Laplace: $(n_i + \beta) / (\sum_j n_j + I \cdot \beta)$	compute_log_q laplace mode matches the formula	✓
Support raw vs. Laplace (warning §3.1)	three modes available: laplace / raw / sqrt	✓
Cache + invalidation when the train split changes	SHA1 signature over (mode, β , n_{items} , $\sum \text{counts}$)	✓
Independent unit tests for the estimator	10 tests (T1–T10) + 1 slow test	✓
enable_logq=False $\Rightarrow$ bit-for-bit identical to rev45	f_simple branch preserved; default flag = 0	✓
Fail-fast when the log_q buffer is all zeros	RuntimeError if <code>log_q.abs().sum()==0</code>	✓
LogQ applies to the item branch only, NOT the user branch	item apply_logq=True, user apply_logq=False	✓
Two extra safety hyperparameters (logq_scale, logq_clip) against float32 overflow	present + clip ± 5.0 , scale sweepable	✓
Five CLI flags (enable_logq, logq_mode/beta/scale/clip)	added to parser.py	✓

3 Compliance Matrix (Code Level)

4 Summary

- **Code level:** $\sim 100\%$ — conformant to the spec, including the safety details.
- **Wave 1 execution level: not met** — because M1.5 has not been run: the notebook does not enable `--enable_logq`, there is no pytest log, no backward-compatibility smoke test, and no scale sweep.

Remaining work to reach 100% Wave 1:

1. Add a notebook cell/variant that passes `--enable_logq 1 --logq_mode laplace --logq_beta 1.0 --logq_scale {sweep}`.
2. Attach the 10/10 pytest log.
3. Run the backward-compatibility diff.
4. Run the 24-run sweep and report the M1.5 gate ($R@20 \geq 0.0925$, with no seed below 0.0890).

5 Appendix: M1.5 Sweep Driver Notebook Cell

The cell below executes the M1.5 sweep over the matrix `logq_scale` $\times$ `logq_mode` $\times$ `seed`. It calls `train.py` once per configuration, parses the `BEST_Test_*` metrics from each run, aggregates per-(mode, scale) means and standard deviations across seeds, and evaluates the M1.5 gate.

Paste it as a new cell at the end of the notebook (it reuses DAMPS_DIR, dataset, and PYTHON_EXE defined in the earlier cells).

Listing 1: M1.5 sweep driver: $\logq_scale \times \logq_mode \times seed$.

```

1  # =====
2  # Section 8.2 -- Wave 1 / M1.5 sweep driver
3  # Matrix: logq_scale x logq_mode x seed (calls train.py per config)
4  # Gate: mean Recall@20 ≥ 0.0925 AND every seed ≥ 0.0890
5  # =====
6  from __future__ import annotations
7
8  import os
9  import re
10 import sys
11 import json
12 import time
13 import random
14 import subprocess
15 from typing import Any
16
17 # ---- 0. Resolve project dirs (reuse earlier-cell globals if present) ----
18 try:
19     _DAMPS_DIR = DAMPS_DIR
20 except NameError:
21     _DAMPS_DIR = os.getcwd()
22 try:
23     _PYTHON = PYTHON_EXE
24 except NameError:
25     _PYTHON = sys.executable
26 try:
27     _DATASET = dataset
28 except NameError:
29     _DATASET = "clothing"
30
31 if os.path.normpath(os.getcwd()) != os.path.normpath(_DAMPS_DIR):
32     os.chdir(_DAMPS_DIR)
33
34 # ---- 1. Sweep matrix definition -----
35 LOGQ_SCALES: list[float] = [0.05, 0.1, 0.3, 1.0] # spec-mandated scale sweep
36 LOGQ_MODES: list[str] = ["laplace"] # add "raw", "sqrt" to widen
37 N_SEEDS: int = 5 # 4 scales x 1 mode x 5 = 20 runs
38 LOGQ_BETA: float = 1.0
39 LOGQ_CLIP: float = 5.0
40
41 # M1.5 acceptance gate (rev54 Section 8.2 / Section 6)
42 GATE_MEAN_RECALL: float = 0.0925 # mean R@20 must reach this
43 GATE_MIN_RECALL: float = 0.0890 # NO seed may fall below this (rollback gate)
44
45 # Reuse the seed list from the Phase 1 sweep if available, else generate.
46 try:
47     SEEDS: list[int] = list(seeds)[:N_SEEDS] # noqa: F821
48     if len(SEEDS) < N_SEEDS:
49         raise NameError
50 except NameError:
51     random.seed(2026)
52     SEEDS = [random.randint(1, 10_000) for _ in range(N_SEEDS)]
53 print(f"[M1.5] seeds = {SEEDS}")
54
55 # ---- 2. Fixed rev45 apc_off_combined backbone flags -----
56 # Identical to the Phase-1 'combined' variant so the ONLY delta is LogQ.
57 BACKBONE_FLAGS: list[str] = [
58     "--dataset", _DATASET,
59     "--temperature", "0.3",
60     "--damps_apc", "0",
61     "--damps_avrf", "0",
62     "--damps_imcf", "1",
63     "--damps_permutation_fft", "1",
64     "--damps_soft_routing", "1",
65     "--damps_momentum", "1",
66     "--damps_data_driven_prior", "1",
67     "--damps_warmup_epochs", "0",
68 ]

```

```

69
70 # ---- 3. Metric parser (matches BEST_Test_* in train.py text logs) ----
71 _PAT = {
72     "recall@20": re.compile(r"BEST_Test_Recall@20[~0-9]*([0-9]*\.[0-9]+)"),
73     "ndcg@20": re.compile(r"BEST_Test_NDCG@20[~0-9]*([0-9]*\.[0-9]+)"),
74 }
75
76 def _parse_metrics(stdout: str) →dict[str, float]:
77     out: dict[str, float] = {}
78     for key, pat in _PAT.items():
79         hits = pat.findall(stdout)
80         if hits:
81             out[key] = float(hits[-1]) # last = best-checkpoint snapshot
82     return out
83
84 # ---- 4. Run one (mode, scale, seed) configuration -----
85 def run_one(mode: str, scale: float, seed: int) →dict[str, Any]:
86     cmd: list[str] = [
87         _PYTHON, "train.py",
88         *BACKBONE_FLAGS,
89         "--seed", str(seed),
90         # ---- Wave 1 LogQ activation (the delta vs rev45) ----
91         "--enable_logq", "1",
92         "--logq_mode", mode,
93         "--logq_beta", str(LOGQ_BETA),
94         "--logq_scale", str(scale),
95         "--logq_clip", str(LOGQ_CLIP),
96     ]
97     t0 = time.time()
98     proc = subprocess.run(cmd, capture_output=True, text=True, cwd=_DAMPS_DIR)
99     dt = time.time() - t0
100     log = proc.stdout + "\n" + proc.stderr
101     metrics = _parse_metrics(log)
102     ok = ("recall@20" in metrics) and (proc.returncode == 0)
103     if not ok:
104         print(f" [WARN] mode={mode} scale={scale} seed={seed} "
105               f"rc={proc.returncode} →last 400 chars:\n{log[-400:]}")
106     return {
107         "mode": mode, "scale": scale, "seed": seed,
108         "recall@20": metrics.get("recall@20"),
109         "ndcg@20": metrics.get("ndcg@20"),
110         "runtime_s": round(dt, 1), "ok": ok,
111     }
112
113 # ---- 5. Execute the full matrix -----
114 all_runs: list[dict[str, Any]] = []
115 total = len(LOGQ_MODES) * len(LOGQ_SCALES) * len(SEEDS)
116 i = 0
117 print("=" * 72)
118 print(f"M1.5 SWEEP: {total} runs "
119       f"({len(LOGQ_MODES)} mode x {len(LOGQ_SCALES)} scale x {len(SEEDS)} seed)")
120 print("=" * 72)
121 for mode in LOGQ_MODES:
122     for scale in LOGQ_SCALES:
123         for seed in SEEDS:
124             i += 1
125             print(f"[{i}/{total}] mode={mode} scale={scale} seed={seed} ...",
126                   flush=True)
127             all_runs.append(run_one(mode, scale, seed))
128
129 # ---- 6. Aggregate per (mode, scale) and evaluate the gate -----
130 def _mean(xs: list[float]) →float:
131     return sum(xs) / len(xs) if xs else float("nan")
132
133 def _std(xs: list[float]) →float:
134     if len(xs) < 2:
135         return 0.0
136     m = _mean(xs)
137     return (sum((x - m) ** 2 for x in xs) / (len(xs) - 1)) ** 0.5
138
139 summary: list[dict[str, Any]] = []
140 for mode in LOGQ_MODES:
141     for scale in LOGQ_SCALES:

```

```

142     rs = [r["recall@20"] for r in all_runs
143           if r["mode"] == mode and r["scale"] == scale
144           and r["recall@20"] is not None]
145     ns = [r["ndcg@20"] for r in all_runs
146           if r["mode"] == mode and r["scale"] == scale
147           and r["ndcg@20"] is not None]
148     if not rs:
149         continue
150     mean_r, min_r = _mean(rs), min(rs)
151     passed = (mean_r ≥ GATE_MEAN_RECALL) and (min_r ≥ GATE_MIN_RECALL)
152     summary.append({
153         "mode": mode, "scale": scale, "n": len(rs),
154         "recall_mean": round(mean_r, 4), "recall_std": round(_std(rs), 4),
155         "recall_min": round(min_r, 4),
156         "ndcg_mean": round(_mean(ns), 4), "ndcg_std": round(_std(ns), 4),
157         "M1.5_pass": passed,
158     })
159
160 # ---- 7. Report -----
161 print("\n" + "=" * 72)
162 print("M1.5 GATE REPORT (mean R@20 ≥ 0.0925 AND min R@20 ≥ 0.0890)")
163 print("=" * 72)
164 hdr = f"{'mode':<9}{'scale':>7}{'n':>4}{'R@20 mean':>12}{'+/-':>9} \
165       f"{'R@20 min':>11}{'NDCG mean':>11}{'PASS':>7}"
166 print(hdr)
167 print("-" * len(hdr))
168 for s in summary:
169     print(f"{'s['mode']':<9}{'s['scale']':>7}{'s['n']':>4}"
170          f"{'s['recall_mean']':>12.4f}{'s['recall_std']':>9.4f}"
171          f"{'s['recall_min']':>11.4f}{'s['ndcg_mean']':>11.4f}"
172          f"{'(YES if s['M1.5_pass'] else 'no')':>7}")
173
174 any_pass = any(s["M1.5_pass"] for s in summary)
175 print("\n[M1.5 VERDICT]",
176       "PASS -- at least one (mode,scale) cleared the gate; "
177       "promote that config to Wave 2 (M1 / SimGCL)."
178       if any_pass else
179       "FAIL -- no (mode,scale) cleared the gate; "
180       "inspect logq_scale/clip before proceeding.")
181
182 # ---- 8. Persist raw + summary for the audit trail -----
183 out_dir = os.path.abspath(os.path.join(".", _DATASET))
184 os.makedirs(out_dir, exist_ok=True)
185 with open(os.path.join(out_dir, "m15_logq_sweep_runs.json"), "w") as f:
186     json.dump(all_runs, f, indent=2)
187 with open(os.path.join(out_dir, "m15_logq_sweep_summary.json"), "w") as f:
188     json.dump(summary, f, indent=2)
189 print(f"\nSaved → {out_dir}/m15_logq_sweep_runs.json")
190 print(f"Saved → {out_dir}/m15_logq_sweep_summary.json")

```

Usage notes.

- The cell assumes the earlier environment-setup cell has defined DAMPS_DIR, PYTHON_EXE, and dataset; if not, it falls back to sensible defaults.
- The BACKBONE_FLAGS list reproduces the rev45 apc_off_combined backbone so that the *only* delta versus the validated baseline is the LogQ activation, preserving causal attributability for the M1.5 ablation.
- To widen the sweep into the full 24-run matrix mentioned in the audit, add "raw" (and optionally "sqrt") to LOGQ_MODES; the gate logic and reporting scale automatically.
- Adjust the regular expressions in _PAT if your train.py prints metric keys under a different label than BEST_Test_Recall@20.